

### Question Booklet

- The final exam contains 9 PROBLEMS and is scheduled for 4 HOURS.
- The maximum amount of points you can earn is 148.
- Please verify if your question booklet consists of 7 PAGES (plus 4 pages with supplementary lecture slides) that are printed legibly, else contact your tutor immediately.
- The exam questions address both the theory and exercise parts of the lecture. For questions marked with an asterisk (\*), supplementary lecture slides are provided at the end of this booklet.
- No electronic devices (such as calculators, notebooks, cell phones, PDAs) are allowed.
- Answers without sufficient details are void (e.g.: you can't just say "yes" or "no" as the answer).
- Only the answer booklet will be collected, please do not use the question booklet for answering the questions.

FROM ERM'S TO RELATIONAL DESIGNS

25 POINTS

**Problem 1.** Consider the following properties, relationships, and constraints that describe the 5 entity sets *Part*, *Supplier*, *Customer*, *Country* and *Region* of an online retailer.

- Each of the 5 entity sets has a separate key attribute by which each of the contained entities can be uniquely identified.
- A *Part* has a *name*, a *type* and a suggested *retail price*. Each part uniquely refers to a *Supplier* who is the only retailer of that part.
- A *Supplier* has a *name*, an *address* and refers to a single *Country* where the supplier is located.
- A *Customer* has a *name*, an *address* and refers to a single *Country* where the customer is located.
- A *Country* has a *name* and refers to a single *Region* in which the country is located.
- A *Region* has a *name* and a *type*.
- An *order* is a ternary relationship that involves one or more *Part* entities, exactly one *Supplier* and exactly one *Customer*. Each order is placed at a specific *date*, and it stores a specific *negotiated price* which the customer has negotiated with the supplier for each individual part that is involved in such an order.

Please answer the following questions.

- (a) (\*) Provide an E/R diagram according to the Entity-Relationship-Model (ERM) which captures the above description. Make sure that your E/R diagram properly includes all the *keys* and *multiplicities* indicated in this description. Also indicate for which of the relationships in your E/R diagram a *referential integrity* constraint makes sense.

8 POINTS

- (b) (\*) Translate the above description into a corresponding relation schema in SQL by providing CREATE TABLE statements for each of the above entity sets. Introduce a separate table for a relationship only if this is absolutely necessary to avoid redundancies. Clearly indicate the PRIMARY KEY and FOREIGN KEY dependencies that hold for your relation schema. Also translate the possible referential integrity constraints (if any) for this relation schema to corresponding constraints in SQL.

(Note: you may either use the E/R diagram you provided in (a) or directly start from the above description to answer this question.)

8 POINTS

- (c) Formulate a constraint in SQL for the following condition: “each order may only involve parts that are provided by the same supplier”

4 POINTS

- (d) (\*) Determine whether the relation schema you provided in (b) is in *Boyce-Codd Normal Form* (BCNF). You may assume that the only non-trivial functional dependencies that hold for your relations are those implied by the keys of your relation schema. Recall that a key uniquely determines all the remaining attributes of a relation. Please justify your answer.

5 POINTS

ANALYTICAL QUERIES & INDEXES

12 POINTS

**Problem 2.** Suppose that an analyst is interested in evaluating the amount of orders that took place for an online retailer over the relation schema that is described in Problem 1.

- (a) Create a materialized view (by using a single CREATE TABLE statement in SQL) for the following analytical query:

“Select the sum of the *negotiated price* values of all orders grouped by both the *name* of the *Supplier* and the *date* of the *order*, where the *name* of the *Region* equals *Flanders*.”

6 POINTS

- (b) Provide an index (or a combination of multiple indexes by using appropriate CREATE INDEX statements in SQL) to accelerate queries that select the aggregated price by a given *name* of a *Supplier* and a given *date* of an *order* over the materialized view you created in (a).

4 POINTS

- (c) Change the analytical query you created in (a), such that we additionally also see the aggregated difference between the suggested *retail price* and the *negotiated price*.

2 POINTS

MORE ABOUT CONSTRAINTS IN SQL

12 POINTS

**Problem 3.** To answer the following questions, consider the different policies which can be used in combination with a FOREIGN KEY definition for handling updates and deletions as they were discussed in the lecture.

- (a) Given an example (based on at least *three relation instances*) for the usage of a FOREIGN KEY with an ON DELETE CASCADE policy. Your example should explicitly show the contents of your relation instances before and after a deletion operation that leads to a cascading deletion over the three (or more) relation instances.

6 POINTS

- (b) Which of the three policies REJECT, CASCADE and SET NULL could be implemented with

(i) an ASSERTION, and

(ii) a TRIGGER

instead of specifying these policies in combination with a FOREIGN KEY?

6 POINTS

**FUNCTIONAL VS. MULTIVALUED DEPENDENCIES & KEYS**

**18 POINTS**

**Problem 4.** Which of the following statements are true? Briefly justify each of your answers.

- (a) If there exist only trivial functional dependencies for a relation schema, then every attribute of that relation schema is also a key of that relation schema.  
**3 POINTS**
- (b) If there exist only trivial functional dependencies for a relation schema, then the only key of that relation schema is the set of all its attributes.  
**3 POINTS**
- (c) Every key of a relation schema is also a superkey of that relation schema.  
**3 POINTS**
- (d) Every superkey of a relation schema is also a key of that relation schema.  
**3 POINTS**
- (e) Every functional dependency is also a multivalued dependency.  
**3 POINTS**
- (f) Every multivalued dependency is also a functional dependency.  
**3 POINTS**

**NORMAL FORMS**

**12 POINTS**

**Problem 5.** Please provide a relation schema, including respective constraints in the form of functional and/or multivalued dependencies, for each of the cases in (a)–(c), or argue why this may not be possible.

- (a) (\*) Provide an example for a relation schema that is in 3rd Normal Form (3NF) but not in Boyce-Codd Normal Form (BCNF).  
**4 POINTS**
- (b) (\*) Provide an example for a relation schema that is in Boyce-Codd Normal Form (BCNF) but not in 4th Normal Form (4NF).  
**4 POINTS**
- (c) (\*) Provide an example for a relation schema that is in 4th Normal Form (4NF) but not in Boyce-Codd Normal Form (BCNF).  
**4 POINTS**

*Note: It is sufficient to provide the relation schemas in the form:  $R(A, B, C)$  with  $A \rightarrow B, C$ , where the former denotes the set of attributes associated with  $R$  and the latter denotes the set of functional and/or multivalued dependencies that hold for  $R$ . Please illustrate your answer by also showing an instance of  $R$  (in case this exists).*

## SQL QUERIES

16 POINTS

**Problem 6.** Consider the relation instances **Departments**, **Projects**, **Employees** and **Assignments** as they are depicted below.

| dept_id | name            |
|---------|-----------------|
| 1       | Marketing       |
| 2       | Finance         |
| 3       | Human Resources |
| 4       | R & D           |
| 5       | Management      |

Table 1: **Departments**

| project_id | name                       |
|------------|----------------------------|
| 1          | Viral Advertising Campaign |
| 2          | Bus Stop Billboards        |
| 3          | Recruitment Drive          |
| 4          | Brain Drain                |
| 5          | The Ultimate Product       |

Table 2: **Projects**

| emp_id | name        | salary | gender | dept_id |
|--------|-------------|--------|--------|---------|
| 1      | Bob Roberts | 26000  | M      | 4       |
| 2      | Bill Reilly | 49000  | M      | 5       |
| 3      | Jim Robbins | 28000  | M      | 3       |
| 4      | Jill Doe    | 38000  | F      | 5       |
| 5      | Jane Jones  | 25000  | F      | 1       |
| 6      | Hank Brown  | 35000  | M      | 4       |
| 7      | Joan Warner | 32000  | F      | 2       |
| 8      | Tom Bull    | 39000  | M      | 4       |
| 9      | Ron Little  | 29000  | M      | 1       |

Table 3: **Employees**

| project_id | emp_id |
|------------|--------|
| 1          | 5      |
| 1          | 7      |
| 2          | 9      |
| 2          | 2      |
| 3          | 3      |
| 4          | 3      |
| 4          | 8      |
| 5          | 1      |
| 5          | 6      |

Table 4: **Assignments**

Please translate the following queries to SQL. Notice that some queries may yield an empty result.

- Select the names of all employees currently working on more than one project.  
4 POINTS
- Select the average salary of employees working on the “Brain Drain” project.  
4 POINTS
- Select the names of all projects to which no employees are assigned.  
4 POINTS
- Select the names of projects that have more male employees than female employees assigned.  
4 POINTS

## RELATIONAL ALGEBRA &amp; DATALOG AS QUERY LANGUAGES

16 POINTS

**Problem 7.** Consider again the relation instances as they are depicted for Problem 6.

- Translate the queries formulated in Problem 6 (a) and (b) to *Relational Algebra* and to *Datalog*, respectively, or argue why such a translation may not exist.  
4×4 POINTS

## STORAGE AND INDEXING

12 POINTS

**Problem 8.** Consider the instance **R** of a relation schema  $R(A, B)$  as it is depicted in Table 5. Suppose that we would like to organize the records that are stored under this relation schema in a sequential file. Further assume that we would like to create two index structures to support an efficient access to these records by the values of the  $A$  and  $B$  attributes, respectively. We intend to do this by creating

- a primary index on the  $A$  attribute, and
- a secondary index on the  $B$  attribute.

Please solve the following questions.

- (a) Index the values of the  $A$  attribute using a  $B^+$ -tree index of order  $n = 3$ . You need to show only the final configuration of the  $B^+$ -tree after all values have been inserted.  
(Note: Your solution must properly handle the duplicate values under the  $A$  attribute. That is, for every value of  $A$ , your index must be able to retrieve all the records that match that value.)

6 POINTS

- (b) Consider again the instance **R** that is depicted in Table 5. This time, index the values of the  $B$  attribute using either the *extensible* or the *linear hashing scheme* presented in the lecture. Assume that we are using a fixed block size of  $b = 2$ , and show all the parameters used by the final configuration of your hashing scheme.

(Note: To simplify the solution, the values of  $B$  are depicted already in binary format. Also here, indicate how we can handle the two duplicate values that occur at the end of the  $B$  list.)

6 POINTS

| $A$ | $B$  |
|-----|------|
| 0   | 0001 |
| 9   | 0010 |
| 8   | 0011 |
| 7   | 0100 |
| 6   | 0101 |
| 5   | 0110 |
| 2   | 0111 |
| 1   | 1000 |
| 3   | 1001 |
| 4   | 1010 |
| 1   | 0001 |
| 2   | 0010 |

Table 5: **R**

XML-DOCUMENTS, XML-SCHEMA & XPATH

25 POINTS

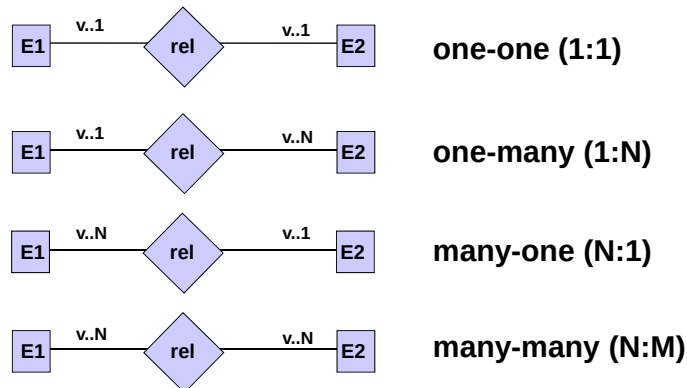
**Problem 9.** Consider the following description of a data structure for encoding a *directed graph* in XML format.

- A *directed graph* consists of both a set of *vertices* and a set of *directed edges*.
  - Every vertex has a unique *name*.
  - Every directed edge has a unique *name*.
  - Every directed edge consists of a *source* and a *target* vertex, which each uniquely refer to the name of the respective vertex.
  - Not all vertices stored in the graph need to occur as either the source or the target vertex of an edge.
- (a) (\*) Provide an *XML-Schema file* that properly specifies the constraints formulated above when storing a directed graph as an XML document.  
5 POINTS
- (b) Provide a *well-formed XML document* that stores the directed graph  $G = (V, E)$  with vertices  $V = \{a, b, c, d, e, f\}$  and edges  $E = \{(a, b), (b, c), (c, d), (d, e), (e, a)\}$ . Your XML document should also be *valid* with respect to your XML-Schema file.  
5 POINTS
- (c) Can we additionally also express the condition that our directed graph should be *acyclic* in XML-Schema? Briefly justify your answer.  
3 POINTS
- (d) Formulate the following queries in XPath. The queries should be specific to the structure of the XML document you provided in (b).
- (i) Find all vertices that are contained in the directed graph.
  - (ii) Find all edges where  $a$  or  $b$  is the source.
  - (iii) Find all edges that do not have  $c$  as their target.
  - (iv) Find all vertices that do not only point to vertex  $c$  (but to at least one other vertex).

4×3 POINTS

## Multiplicities of Relationships in ERM

$$v \in \{0,1\}$$



## Summary of Translations

- **Classes/interfaces** in ODL and **entity sets** in ERM are each translated by creating a new relation.
  - All **attributes** (incl. structs and collection types in ODL) are taken over.
  - **Multi-valued attributes** are flattened by creating a new tuple for each combination of values stored in the multi-valued attributes.
  - **Weak entity sets** and **subclasses** take over the key attributes of their key-supplying and super-class(es), respectively; key and foreign-key references are added.
- **Higher-arity relationships** in ERMs are first translated into binary relationships. (→ Exercise Sheet #1)
- **1:1 relationships** are each translated by adding foreign-key references to both the source and the target relation.
- **1:N relationships** are each translated by adding a foreign-key reference to the target relation. (and vice-versa)
- **N:M relationships** are each translated by creating a new relation, which takes over the key attributes of the source and target relation, and by adding foreign-key references to both the source and the target relation.



## Defining Foreign Keys

```
CREATE TABLE MovieExec (  
  name CHAR(30) UNIQUE,  
  address VARCHAR(255),  
  certN INT PRIMARY KEY,  
  netWorth INT);
```

SQL has two options for defining a foreign key of a relation:

```
CREATE TABLE Studio (  
  name CHAR(30) PRIMARY KEY,  
  address VARCHAR(255),  
  presCertN INT REFERENCES MovieExec(certN));
```

This is only possible if the foreign key contains exactly one attribute.

```
CREATE TABLE Studio (  
  name CHAR(30) PRIMARY KEY,  
  address VARCHAR(255),  
  presCertN INT,  
  FOREIGN KEY (presCertN) REFERENCES MovieExec(certN));
```

This allows any combination of attributes to be a foreign key.



## A Complete Set of Inference Rules

### Armstrong's Axioms:

The following three axioms allow for deriving any FD that follow from a given set of FD's.

#### 1. Reflexivity:

If  $\{B_1, B_2, \dots, B_m\} \subseteq \{A_1, A_2, \dots, A_n\}$ ,  
then  $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ . ( $\rightarrow$  "trivial FDs")

#### 2. Augmentation:

If  $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ ,  
then  $A_1, A_2, \dots, A_n, C_1, C_2, \dots, C_k \rightarrow B_1, B_2, \dots, B_m, C_1, C_2, \dots, C_k$ ,  
for any set of attributes  $C_1, C_2, \dots, C_k$ . ( $\rightarrow$  "non-trivial FDs")

#### 3. Transitivity:

If  $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$  and  $B_1, B_2, \dots, B_m \rightarrow C_1, C_2, \dots, C_k$ ,  
then  $A_1, A_2, \dots, A_n \rightarrow C_1, C_2, \dots, C_k$ . ( $\rightarrow$  "closure algorithm")



## Normal Forms of Relations

In the following, let  $X$  be a set of attributes and  $Y$  be a single attribute of a relation  $R$ , and let  $F$  be a given set of FD's on  $R$ .

$R$  is in **First Normal Form** (1NF), if all attribute values of every tuple are atomic. All the relations we consider are in 1NF.

$R$  is in **Second Normal Form** (2NF), if for every non-trivial FD  $X \rightarrow Y$  in  $F$  it holds that  $X$  is not a proper subset of a key of  $R$  or  $Y$  is an attribute of a key of  $R$ .

$R$  is in **Third Normal Form** (3NF), if for every non-trivial FD  $X \rightarrow Y$  in  $F$  it holds that  $X$  is a superkey of  $R$  or  $Y$  is an attribute of a key of  $R$ .

$R$  is in **Boyce-Codd Normal Form** (BCNF), if for every non-trivial FD  $X \rightarrow Y$  in  $F$  it holds that  $X$  is a superkey of  $R$ .



## Fourth Normal Form

A MVD  $A_1, A_2, \dots, A_n \twoheadrightarrow B_1, B_2, \dots, B_m$  over a relation  $R$  is called **non-trivial** if

- none of the  $B$ 's is among the  $A$ 's;
- not all of the attributes of  $R$  are among the  $A$ 's and  $B$ 's.

A relation is in **Fourth Normal Form** (4NF), if for every non-trivial MVD  $A_1, A_2, \dots, A_n \twoheadrightarrow B_1, B_2, \dots, B_m$  over  $R$  it holds that  $\{A_1, A_2, \dots, A_n\}$  is a superkey of  $R$ .

→ Note that the definition of “keys” and “superkeys” still depends on FD's only!



## Example: An `xs:complexType` for Bars

```
<xs:complexType name = "barType">
  <xs:sequence>
    <xs:element name = "NAME"
      type = "xs:string"
      minOccurs = "1" maxOccurs = "1" />
    <xs:element name = "BEER"
      type = "beerType"
      minOccurs = "0" maxOccurs =
        "unbounded" />
  </xs:sequence>
</xs:complexType>
```

Like \* in  
a DTD



## Example: Multi-Attribute Keys and Foreign Keys

```
...
<xs:element BARS_AND_DRINKERS>
  ...
  <xs:key name = "barKey">
    <xs:selector xpath = "./BAR" />
    <xs:field xpath = "@owner" />
    <xs:field xpath = "@city" />
  </xs:key>
  <xs:keyref name = "barRef" refer = "barKey">
    <xs:selector xpath = "./DRINKER/FREQ" />
    <xs:field xpath = "@ownerRef" />
    <xs:field xpath = "@cityRef" />
  </xs:keyref>
</xs:element>
<xs:element name = "BAR"> ... </xs:element>
<xs:element name = "DRINKERS"> ... </xs:element>
...
```

The two attributes `owner` and `city` together constitute a multi-attribute key for `BAR` elements.

The two attributes `ownerRef` and `cityRef` constitute a multi-attribute foreign key for `DRINKER` elements that refers to `barKey`. Notice that the attribute names may be different from the key attribute names of `BAR`; only their order matters.

